

CS321 Programming Language
Task Activity

Name: Mark Renier B. Fostanes
Section: BSCS 3a2

Instructions:

1. Analyze each problem carefully before answering.
2. Some questions require reasoning about program execution.
3. Provide explanations where required.

1. Consider the following program.

```
def transform(data):  
    data = data + [100]  
    return data  
  
numbers = [10, 20, 30]  
result = transform(numbers)  
  
print(numbers)  
print(result)
```

Tasks:

1. Predict the exact output of the program.
2. Explain why the original list was or was not modified.
3. Identify which operation creates a new object.
4. Explain the role of mutability in this scenario.

ANS:

1. Output:
 [10, 20, 30]
 [10, 20, 30, 100]
2. The original list was not modified because `data + [100]` creates a new list instead of changing the old one in place.
3. The operation that creates a new object is: `data + [100]`
4. Mutability matters here because lists are mutable, but the code used `+` and not `append()`, so a new list was returned.

2. Analyze the following code.

```
record = ("Alice", "Bob", 2025)  
  
record[0].append("Charlie")  
  
print(record)
```

Tasks:

1. Predict the output.

2. Explain why the tuple allowed modification.
3. Identify which element is mutable.
4. Explain how this relates to shallow immutability.

ANS:

1. Output:

```
(['Alice', 'Bob', 'Charlie'], 2025)
```
 2. The tuple allowed the change because the tuple itself did not change. Only the list inside it changed.
 3. The mutable element is the list ["Alice", "Bob"].
 4. This is called shallow immutability. The tuple is immutable on the outside, but inner mutable objects can still be changed.
3. The following program was written incorrectly.

```
for i in range(1, 15):
```

```
    if i % 2 == 0:  
        continue
```

```
    if i == 11:  
        break
```

```
    if i % 5 == 0:  
        pass
```

```
    print(i)
```

Tasks:

1. Write the exact output sequence.
2. Explain how continue, break, and pass affected execution.
3. Describe the order in which conditions are evaluated.

ANS:

1. Output sequence:

```
1  
3  
5  
7  
9
```
2. Continue skips even numbers. break stops the loop when i == 11. pass does nothing, so 5 is still printed.
3. Order of evaluation: first check even, second check 11, third check divisible by 5, then print.

<https://www.youtube.com/watch?v=cPKPDpYqlwE>

4. The following program attempts to compute the average of positive numbers.

```
numbers = [5, -2, 8, 0, 3, -7, 6]
```

```

total = 0
count = 0

for n in numbers:
    if n > 0:
        total += n
        count += 1

average = total / count

print("Average:", average)

```

Tasks:

1. Identify the logical error.
2. Explain why the result is incorrect.
3. Rewrite the program so it correctly computes the average of only positive numbers.

ANS:

1. The logical error is that `count += 1` is outside the `if` block, so it counts all numbers instead of only positive numbers. Positive numbers are 5, 8, 3, and 6. Their sum is 22 and their correct count is 4, so the correct average is 5.5.

2. The wrong code uses `22 / 7`, which gives about 3.14.

3. Correct code:

```

numbers = [5, -2, 8, 0, 3, -7, 6]

total = 0
count = 0

for n in numbers:
    if n > 0:
        total += n
        count += 1

average = total / count
print("Average:", average)

```

Correct output: Average: 5.5

5. Given the following function:

```

def evaluate(x):

    if x > 0:
        if x % 2 == 0:
            return "A"
        elif x % 3 == 0:
            return "B"
        else:
            return "C"

```

```
elif x == 0:  
    return "D"
```

```
else:  
    if x % 2 == 0:  
        return "E"  
    else:  
        return "F"
```

Determine the output for:

```
evaluate(6)  
evaluate(9)  
evaluate(7)  
evaluate(-4)  
evaluate(-5)  
evaluate(0)
```

Explain each result.

ANS:

```
evaluate(6) -> "A"  
evaluate(9) -> "B"  
evaluate(7) -> "C"  
evaluate(-4) -> "E"  
evaluate(-5) -> "F"  
evaluate(0) -> "D"
```

Reason: A is for positive even, B is for positive divisible by 3 but not even, C is for other positive numbers, D is for zero, E is for negative even, and F is for negative odd.